

Lab 3 — Architectural Justification

Local Sports Tournament & Bracket Manager (Problem Statement #54)

Architecture Selection

We chose the Layered Architecture for the Local Sports Tournament & Bracket Manager System. The system is cleanly separated into a Presentation Layer (User Interface Component), a Business Layer (Bracket Manager, Score Manager and Standings Service Components) and a Data Layer (Database Component).

Reason 1 — Clear separation of concerns matches the domain

The system has three naturally distinct concerns: showing brackets and standings to users, running tournament logic (bracket generation, score validation, tiebreakers), and persisting teams, matches and scores. Mapping each concern to its own layer keeps responsibilities isolated — the Standings Service can be rewritten to add a new tiebreaker rule without touching the UI or the database schema, which is realistic for a tournament platform that will evolve across sports.

Reason 2 — Right-sized complexity for a local tournament

A microservices approach would introduce network latency, service discovery and distributed-data headaches that are overkill for a single local tournament deployment. A pure client-server design would collapse the business logic into one tier, making tiebreaker and standings rules harder to maintain. The layered style gives us structure without the operational cost of microservices and avoids the monolithic coupling of flat client-server designs.

Security Advantage

NFR-002 requires that only authorised Tournament Directors can enter or update scores. In the layered design, the UI cannot reach the Database directly — every write must pass through the Business Layer, where the Score Manager Component authenticates the caller and validates the request before invoking the Database Interface. This single enforcement point prevents a tampered client from bypassing authorisation and writing straight to the data store.

Performance Benefit

NFR-001 requires standings and leaderboard updates within 200 ms of score submission. Because all business components run in-process within one layer, the Score Manager calls the Standings Service and the Database Component through direct method calls rather than remote network hops. This keeps the end-to-end recalculation path short and predictable, comfortably within the 200 ms budget under local tournament load.